Plano de Verificação

SoC riscv32-ext-coprocessor

Core RV32I, coprocessador sistólico 8×8, DMA e barramentos

DUTs: `rv32_core` · `coproc_top` · DMA · timer · barramentos · `soc_top`

Equipe do projeto riscv32-ext-coprocessor

CEPEDI — CI Digital

Programa de Desenvolvimento de Competências em Sistemas Digitais

Documento	Plano de Verificação (versão inicial)
Versão	0.1
Data	13/11/2026
Base	READMEs de `rtl/core`, `rtl/coproc`, `rtl/dma`, `rtl/top`, `rtl/periph` e `sw`; RTL do core (`rv32_*`) e do coprocessador (`coproc_*`) em 13/11/2026

Sumário

1 Introdução **2**

1.1 Visão geral do projeto 2

1.2 Descrição dos DUTs 2

1.3 Objetivos e metas da verificação 4

1.4 Ambiente e ferramentas 5

2 Plano de teste **6**

2.1 Features a serem verificadas 6

2.1.1 Pontos de atenção levantados na revisão do RTL 10

2.2 Estratégias de inserção de estímulos 11

2.3 Modelo de cobertura 12

2.4 Resumo dos testes planejados 13

3 Cronograma preliminar **14**

Histórico de revisões

Versão	Data	Descrição
0.1	13/11/2026	Versão inicial: introdução, DUTs, metas, plano de teste, cobertura e cronograma preliminar.

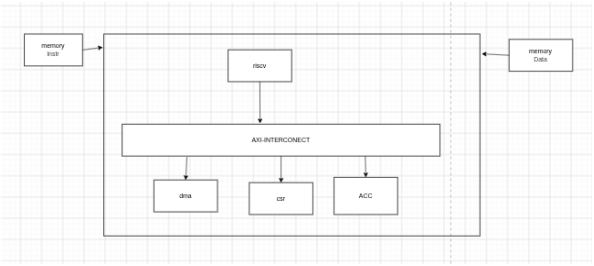


Figura 1: Caption

1 Introdução

1.1 Visão geral do projeto

O projeto é um SoC para ASIC que existe para uma conta: $C = A \cdot B$, com A e B em int8 e C em int32, o núcleo da inferência de redes neurais quantizadas. O trabalho é dividido entre três blocos, cada um com uma única função:

- **Core RV32I** (`rv32_core`): processador próprio de cinco estágios, RV32I + Zicsr, apenas em modo máquina. Roda o programa, configura a DMA e o coprocessador por registradores mapeados em memória e trata as interrupções. Não tem instrução customizada.
- **DMA de 3 canais**: lê A e B da L2 por AXI4 de 128 bits e os entrega como streams AXI4-Stream; recebe C e o grava na L2.
- **Coprocessador** (`coproc_top`): array sistólico 8×8 output-stationary, 64 MACs int8 por ciclo, no seu próprio domínio de clock (`clk_acc_i`). Não tem endereços: consome os streams na ordem em que chegam.

O tráfego é separado em três planos: controle (AXI4-Lite de 32 bits), dados (AXI4 de 128 bits e streams) e eventos (interrupções). A interconexão vem da biblioteca `axi` da PULP (v0.39.11) e das `common_cells` (v1.39.0), em `vendor/`. O RTL do core já está em síntese no Cadence Genus, que também alimenta o estudo comparativo de área $\times f_{\max}$.

Diferente de um projeto com um único algoritmo, este SoC não tem um só modelo de referência. A verificação se apoia em três:

1. o **Spike**, simulador oficial do RISC-V, para o core, instrução a instrução (co-simulação);
2. um **modelo em Python/numpy** da multiplicação de matrizes com aritmética int32 com *wrap*, para o coprocessador e o sistema;
3. **modelos de transação** em SystemVerilog para a DMA, os barramentos e os registradores (preditor de endereços e modelo de registradores UVM, o RAL).

1.2 Descrição dos DUTs

A Figura **??** mostra o SoC e a Tabela 1 os três níveis em que ele é verificado. Cada bloco entra no SoC só depois de passar no seu próprio ambiente.

Core. Pipeline clássico IF, ID, EX, MEM, WB, com duas portas (instrução e dados). O IF/ID não tem registrador de instrução: a memória de instrução mantém a saída enquanto

Tabela 1: Níveis de verificação, DUTs e ambientes.

Nível	DUTs	Ambiente
N1 Unidade	Core: `rv32_alu`, `rv32_decoder`, `rv32_imm_gen`, `rv32_regfile`, `rv32_hazard`, `rv32_branch_pred`, `rv32_pc`, `rv32_csr`, `rv32_load_align`, `rv32_store_align`. Coprocessador: `coproc_mac`, `coproc_pe`, `coproc_skew`, `coproc_pe_array`.	Testbenches SystemVerilog dirigidos (`tb_rv32_*`, `tb_mac`) e o UVM `core_pc`, que já existem; mantidos como regressão rápida.
N2 Bloco	`rv32_core`; `coproc_top` (com `coproc_core` e `coproc_regs`); DMA; timer; pontes OBI→AXI4 e OBI→AXI4-Lite com os interconnects; memórias (Boot ROM, I-TCM, D-TCM, L2).	Um ambiente UVM por bloco em `tb/uvm/`, com agentes reaproveitáveis (OBI, AXI4, AXI4-Lite, AXI4-Stream, IRQ) e o modelo de referência do bloco.
N3 Sistema	`soc_top`.	Programas C e assembly de `sw/tests` carregados nas memórias; monitores passivos dos ambientes de bloco, co-simulação Spike e modelo numpy.

Tabela 2: Mapa de endereços: o que cada porta enxerga.

Base	Tamanho	Região	Busca	`lw/sw`	DMA	Caminho
`0x0000_0000`	4 KiB	Boot ROM, vetor de reset	•	—	—	porta de instrução
`0x1000_0000`	32 KiB	I-TCM: programa	•	—	—	porta de instrução
`0x1001_0000`	32 KiB	D-TCM: dados, pilha, `.rodata`	—	•	—	OBI direto, 1 ciclo
`0x2000_0000`	64 KiB	L2 SRAM: A, B e C	—	•	•	OBI→AXI4
`0x4000_0000`	4 KiB	Registradores do timer	—	•	—	OBI→AXI4-Lite
`0x4000_1000`	4 KiB	Registradores da DMA	—	•	—	OBI→AXI4-Lite
`0x4000_2000`	4 KiB	Registradores do coprocessador	—	•	—	OBI→AXI4-Lite→CDC
Qualquer outro endereço: access fault (`mcause` 1 na busca, 5 num load, 7 num store).						

`instr_req_o` está em 0, e `instr_req_o = ~stall`. O ID lê o banco, decodifica, prediz desvios (para trás tomado, JAL redirecionado em ID) e calcula os seletores de forwarding, que ficam registrados no ID/EX. O EX resolve desvios e JALR e envia o pedido de dados (OBI) com o endereço direto do somador. O MEM é o ponto de commit: CSRs, exceções, interrupções e MRET. O `freeze` congela o pipeline inteiro quando o MEM espera `data_rvalid_i` ou o EX espera `data_gnt_i`. Custos documentados: JAL e desvio para trás, 1 ciclo; erro de predição e JALR, 2; trap e MRET, 3; load ou CSR seguido de uso, 1.

Coprocessador. O software escreve `DIM_M`, `DIM_N`, `DIM_K` e `CTRL.START`. O `coproc_top` leva o início do job (com as dimensões) ao domínio `clk_acc_i` por um handshake de duas fases, e os três streams por FIFOs assíncronas. O `coproc_core` calcula os tiles 8×8 de C em ordem de linha: K passos de dados, 14 passos de dreno com zeros e 16 beats de 128 bits no stream C. O `STATUS.DONE` indica que o último beat saiu do array, não que chegou à L2.

DMA. Três canais fixos: 0 (L2→A, 64 b), 1 (L2→B, 64 b) e 2 (C→L2, 128 b). Cada canal move um bloco 3D: `CNT2` planos de `CNT1` linhas de `LEN` bytes, a partir de $\texttt{ADDR} + p \cdot \texttt{STRIDE2} + r \cdot \texttt{STRIDE1}$. O fim verdadeiro do job é o `DONE` do canal 2, que gera a interrupção externa.

Tabela 3: Estado de cada bloco na data deste plano.

Bloco	RTL	Verificação existente
Core (`rv32_core`)	completo, em síntese	TBs de unidade, UVM do `rv32_pc`; riscv-tests pendentes (marco v0.2)
Coprocessador (`coproc_*`)	núcleo, registradores e topo com CDC	`tb_mac`; modelo Python de ciclo do núcleo; UVM planejado (v0.3)
DMA	não iniciado	UVM planejado (v0.3)
Barramentos	IP da PULP em `vendor/`, sem wrappers	—
Memórias e `prim_`	não iniciado	—
Timer	não iniciado	—
`soc_top`	não iniciado	programas em `sw/tests` (v0.4)

1.3 Objetivos e metas da verificação

Objetivo geral. Mostrar, com evidência mensurável, que cada bloco implementa a sua especificação (os READMEs de `rtl/`) e que o SoC integrado executa corretamente o fluxo de multiplicação de matrizes do `sw/README`, do reset ao fim do job por interrupção. Isso inclui a equivalência do core com o Spike instrução a instrução, a equivalência bit a bit do coprocessador e do sistema com o modelo numpy, e a manutenção desse comportamento na netlist sintetizada.

Objetivos específicos.

O1. Verificar cada unidade isoladamente antes da integração, reaproveitando os testbenches existentes.

O2. Core: conformidade com RV32I + Zicsr em modo M (riscv-tests e riscv-arch-test) e co-simulação com o Spike a cada instrução aposentada, incluindo traps e interrupções.

O3. Core: exercitar a microarquitetura (forwarding, stalls, predição, redirects simultâneos, freeze) com geração aleatória de instruções e latências aleatórias de barramento.

O4. Coprocessador: equivalência bit a bit do stream C com o modelo para todas as dimensões válidas, com backpressure e razões de clock variadas.

O5. DMA: padrões de endereço 3D, bursts AXI4 legais, divisão 128→64 bits e tratamento de erros.

O6. Barramentos e periféricos: decodificação de endereços, pontes non-posted, respostas de erro e timer.

O7. Sistema: programas de `sw/tests` do boot ao fim do job, com polling e com interrupção, incluindo o timer como vigia.

O8. CDC do coprocessador: checagem estrutural e simulação com clocks assíncronos.

O9. Medir e fechar cobertura de código e funcional por bloco, com merge da regressão no IMC.

O10. Equivalência RTL × netlist (Conformal LEC) a cada síntese; simulação gate-level com SDF de um subconjunto quando houver biblioteca do PDK com modelos de simulação.

Tabela 4: Metas quantitativas (critérios de encerramento).

Métrica	Meta
Features do plano (Tab. 5) com teste associado e aprovado	100 %
riscv-tests aplicáveis (`rv32ui-p-*`, `rv32mi-p-*`)	100 % aprovados
riscv-arch-test (RV32I, Zicsr, privilegiado M aplicável) via RISCOF	100 % das assinaturas iguais às do Spike
Divergências na co-simulação Spike na regressão aleatória do core ($\geq$ 500 sementes)	0
Divergências do coprocessador com o modelo numpy ($\geq$ 2000 jobs aleatórios)	0
Programas de `sw/tests` no `soc_top`	100 % aprovados
Cobertura funcional (covergroups da Seção 2.3)	100 % (exclusões justificadas)
Cobertura de linha / bloco, por bloco	$\geq$ 98 %
Cobertura de desvio (branch) e expressão, por bloco	$\geq$ 95 %
Cobertura de FSM: estados e transições	100 %
Cobertura de toggle (portas e registradores de controle)	$\geq$ 90 %
Falhas de asserção (SVA) e de checkers de protocolo na regressão	0
Conformal LEC RTL × netlist dos blocos sintetizados	equivalente
Bugs abertos de severidade alta	0

Fora do escopo. Temporização e área (cobertas pela síntese); potência; verificação física; verificação formal de propriedades além de LEC e CDC estrutural; as extensões M, A e C e os modos S e U; a IP da PULP em si (tratada como confiável: só a sua integração é verificada). O desempenho do array é medido e registrado, mas não é critério de aprovação.

1.4 Ambiente e ferramentas

- **Simulação:** Cadence Xcelium (`xrun`), SystemVerilog, SVA e UVM (a biblioteca que acompanha o Xcelium). Formas de onda no SimVision. Ambientes UVM em `tb/uvm/`,

como a equipe definiu; testbenches de unidade mantidos onde estão.

- **Cobertura:** Cadence IMC (merge de regressão, análise e exclusões).
- **Modelo do core:** Spike (`-isa=rv32i_zicsr`, modo M) ligado ao testbench por DPI-C. A co-simulação é derivada da do Ibex, simplificada, e lê a interface RVFI do core. Será reaproveitada no nível de sistema.
- **Modelo do coprocessador e do sistema:** script Python/numpy que gera A, B e o C esperado (int8×int8→int32 com wrap) nos layouts de painel do `sw/README`, em arquivos lidos pelo scoreboard.
- **Software:** riscv-gnu-toolchain (`-march=rv32i_zicsr -mabi=ilp32`); riscv-tests; riscv-arch-test com RISCOF; riscv-dv (gerador de instruções em SV/UVM, que roda no Xcelium); programas de `sw/tests`.
- **Lint:** HAL (Xcelium) ou `verilator -lint-only`, em todo commit.
- **Complementar:** Conformal LEC com os scripts que o Genus gera; CDC estrutural do `coproc_top` com Jasper CDC, se houver licença; os pacotes de teste da biblioteca `axi` (mestres e escravos aleatórios) para estimular os interconnects.

A Figura 2 mostra os dois ambientes de bloco com mais lógica própria. Os agentes são escritos uma vez e reaproveitados: o agente AXI4-Lite com o RAL serve ao coprocessador, à DMA e ao timer; os agentes AXI4-Stream servem ao coprocessador e à DMA; no `soc_top` todos ficam passivos, só monitorando.

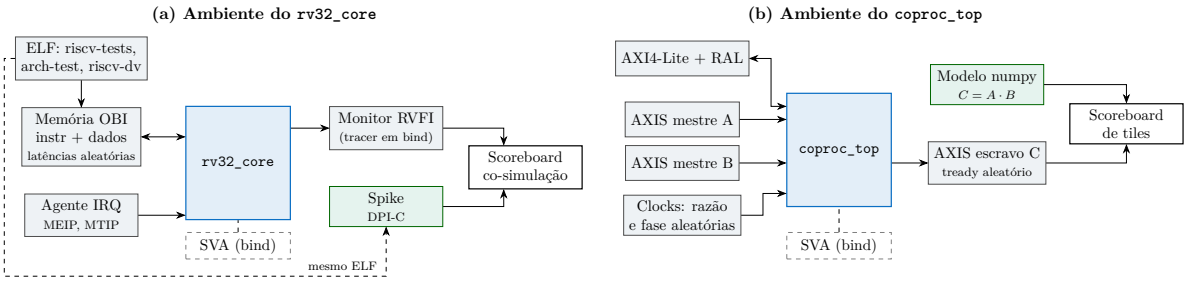


Figura 2: Ambientes de bloco. Cinza: agentes UVM; verde: modelos de referência. As interrupções chegam ao Spike pelo `mip` que o monitor RVFI registra no commit. O ambiente da DMA combina os agentes AXI4-Lite e AXI4-Stream de (b) com um agente escravo AXI4 que modela a L2 e um preditor de endereços; no `soc_top` todos os agentes viram monitores passivos.

Infraestrutura que ainda não existe e é pré-requisito. (i) Um tracer RVFI do core, em `bind`, sem tocar no RTL: `pc`, instrução, `rd` e valor escrito, endereço e dados de memória, trap e interrupção, no ciclo em que a instrução sai do MEM. (ii) Os agentes OBI, AXI4, AXI4-Lite, AXI4-Stream e IRQ. (iii) O script de regressão (`make` com `xrun`, sementes, merge de cobertura). (iv) Os modelos de registradores; o ideal é gerá-los da mesma fonte que o RTL e o header C (ponto P11).

2 Plano de teste

2.1 Features a serem verificadas

A Tabela 5 lista as features extraídas do RTL e dos READMEs. Métodos: **D** = teste dirigido, **R** = aleatório restrito, **A** = asserção, **M** = comparação com modelo de referência, **C** = cobertura

funcional, **G** = netlist (LEC ou gate-level), **F** = checagem estrutural/formal. Prioridade: **A**lta, **M**édia, **B**aixa.

Tabela 5: Features a serem verificadas.

ID	Feature	Comportamento esperado / critério	Método	Pri.
Core: reset e busca				
CORE-01	Reset e boot	Com `rst_ni=0`: PC em `BootAddr`, bits de valid em 0. A primeira busca é em `BootAddr` e nada é escrito antes da primeira instrução.	D, A	A
CORE-02	Contrato da porta de instrução	`instr_req_o = ~stall`; a instrução retida no IF/ID não muda durante um stall; o PC só anda com o IF/ID.	A, R	A
CORE-03	Erro na busca	`instr_err_i` gera access fault na busca (`mcause` 1) sem efeito da instrução.	D, M	A
Core: conjunto de instruções				
CORE-04	Instruções RV32I	Todas as instruções inteiras com resultado igual ao Spike: aritmética, lógica, deslocamentos (0, 31, bits altos de `rs2` ignorados), comparações com e sem sinal, LUI, AUIPC, FENCE como no-op.	D, R, M	A
CORE-05	Loads e stores	LB, LH, LW, LBU, LHU, SB, SH, SW em todos os offsets permitidos; extensão de sinal; byte enables; dado replicado nas lanes.	D, R, M	A
CORE-06	Zicsr	CSRRW/S/C e formas imediatas; `rs1=x0` ou `zimm=0` não escreve; CSR não implementado ou escrita em CSR só de leitura gera instrução ilegal; campos só de leitura de `misa`, `mtval` e `mip` ignoram escrita.	D, R, M	A
CORE-07	Instruções ilegais	Opcodes indefinidos, extensão M, `funct3`/`funct7` inválidos, SYSTEM com `rd` ou `rs1` diferente de 0 geram `mcause` 2. WFI é no-op.	D, R, M	A
CORE-08	Registrador `x0`	Escritas ignoradas, inclusive como destino de load e CSR; nunca é fonte de forwarding.	D, R, M	A
Core: pipeline				
CORE-09	Forwarding	Dependências a 1, 2 e 3 instruções (EX/MEM, MEM/WB, write-through), em `rs1`, `rs2`, dado de store e operando de CSR.	D, R, M, C	A
CORE-10	Load-use e CSR-use	Um ciclo de stall e uma bolha; sem stall quando a instrução não lê o registrador (`rs1_ren`, `rs2_ren`).	D, A, C	A
CORE-11	Predição estática	Desvio para trás previsto tomado, para frente não tomado; JAL redirecionado em ID; erro de predição e JALR corrigidos em EX.	D, R, M, C	A
CORE-12	Prioridade de redirects	Trap > EX > ID quando chegam juntos; redirect de ID espera o fim do stall; redirect de EX repetido durante um freeze não tem efeito.	D, A, C	A
CORE-13	Freeze	`mem_wait` (resposta atrasada) e `req_wait` (grant atrasado) congelam o pipeline sem perder nem duplicar instrução; resposta guardada quando chega durante outro freeze.	D, R, A	A
CORE-14	Custos de ciclo	JAL e desvio para trás: 1 ciclo; erro de predição e JALR: 2; trap e MRET: 3; load-use: 1. Medidos em sequências dirigidas.	D	M
Core: porta de dados (OBI)				

ID	Feature	Comportamento esperado / critério	Método	Pri.
CORE-15	Protocolo OBI	`req` mantido até `gnt`, com endereço, `we`, `be` e `wdata` estáveis; um pedido pendente por vez; latências de `gnt` e `rvalid` de 0 a N ciclos.	A, R	A
CORE-16	Acesso desalinhado	LH, LW, SH e SW desalinhados geram `mcause` 4 ou 6 sem pedido no barramento.	D, M	A
CORE-17	Erro de acesso	`data_err_i` gera `mcause` 5 ou 7; o load não escreve `rd`.	D, M	A
Core: traps e interrupções				
CORE-18	Exceções	Cada causa (0, 1, 2, 3, 4, 5, 6, 7, 11): `mepc` e `mcause` corretos, `mtval=0`, salto para `mtvec`, MIE→MPIE.	D, M	A
CORE-19	Exceções precisas	Nenhuma instrução mais nova que a que gera trap escreve registrador, CSR ou memória; o pedido do EX é descartado quando o MEM gera trap no mesmo ciclo.	D, R, A, M	A
CORE-20	Interrupções	MEI e MTI com `mstatus.MIE` e `mie`; externa tem prioridade sobre o timer; aceitas em MEM, com `mepc` igual à instrução substituída; linhas por nível.	D, R, M, C	A
CORE-21	Interrupção com acesso pendente	Não é aceita sobre um load ou store cujo pedido já saiu ou espera grant (`irq_allowed`); é aceita logo depois.	D, C	A
CORE-22	MRET	MIE←MPIE, MPIE←1, retorno a `mepc`; com interrupção pendente e habilitada, reentra no tratador.	D, M	A
CORE-23	Contadores	`mcycle` conta todo ciclo; `minstret` só instruções aposentadas; uma escrita substitui o incremento daquele ciclo; partes altas.	D, A	M
Coprocessador (`coproc_top`)				
COP-01	Registradores	`CTRL`, `STATUS`, `DIM_M/N/K`: RW, RO e W1C; `START` lido como 0; `WSTRB` respeitado; offset fora do mapa responde SLVERR.	D, R	A
COP-02	Ocupado	Com `BUSY`: `START` e escritas em `DIM_*` ignoradas (OKAY sem efeito); `START` aceito limpa `DONE`; `DONE` ao fim do job.	D, A	A
COP-03	Multiplicação	Stream C igual ao modelo, bit a bit, para M, N múltiplos de 8 e K de 1 a $2^{16}-1$, um ou muitos tiles.	D, R, M	A
COP-04	Formato dos streams	Beat de A = coluna k do painel (lane r); B = linha k do painel (lane c); C em tiles na ordem de linha, cada tile por linhas, 4 int32 por beat, coluna menor nos bits baixos, 16 beats por tile.	D, M	A
COP-05	Valores extremos	-128×-128, 127×-128, zeros, K grande; acumulação int32 com wrap, sem saturação.	D, M	A
COP-06	Backpressure	`tvalid` de A e B com lacunas independentes; `c_tready` baixo por períodos aleatórios; resultado inalterado.	R, M, C	A
COP-07	Jobs consecutivos	O job seguinte começa limpo (clr em diagonal, skew esvaziado pelo dreno), inclusive com outras dimensões.	D, R, M	A
COP-08	Travessia de clock	`clk_acc_i` mais rápido, mais lento, igual e com fase aleatória (razões de 1:4 a 4:1); início e fim do job atravessam uma vez; FIFOs sem perda.	R, A, M, F	A
COP-09	Reset	Reset durante o job em cada estado (carga, dreno, saída); o reset de `clk_acc_i` é sincronizado internamente; operação limpa depois.	D, R	A

ID	Feature	Comportamento esperado / critério	Método	Pri.
COP-10	Dimensões fora do padrão	M ou N não múltiplos de 8, M ou $N < 8$, `DIM_K=0`: comportamento conforme a decisão do ponto P1.	D	A
COP-11	Desempenho	Ciclos por tile medidos: $K + 14$ (dreno) +16 (saída) no mínimo. Registrado, sem critério de aprovação.	D	B
DMA				
DMA-01	Registradores	Por canal (base + `0x20`·canal): `ADDR`, `LEN`, `CNT1`, `STRIDE1`, `CNT2`, `STRIDE2`, `CTRL`, `STATUS`; `START` autolimpante; `DONE` e `ERR` W1C; escritas ignoradas com `BUSY`.	D, R	A
DMA-02	Endereços 3D	Linha r do plano p em `ADDR` $+ p \cdot$ `STRIDE2` $+ r \cdot$ `STRIDE1`, `LEN` bytes; stride 0 repete o dado; `CNT`=1.	D, R, M	A
DMA-03	Bursts AXI4	INCR que nunca cruzam 4 KiB, tamanhos e `WSTRB` corretos nas bordas; leituras dos canais 0 e 1 em paralelo com a escrita do canal 2.	R, A, M	A
DMA-04	Streams	Canais 0 e 1 dividem cada beat de 128 b em dois de 64 b, metade baixa primeiro; canal 2 recebe 128 b; backpressure nos dois sentidos.	D, R, M	A
DMA-05	Erros	Resposta SLVERR ou DECERR marca `ERR` e para o canal, sem afetar os outros.	D	A
DMA-06	Interrupção	`irq_o` em 1 enquanto algum canal com `IRQ_EN` tiver `DONE` ou `ERR`; cai com W1C ou com `IRQ_EN=0`.	D, A	A
DMA-07	Programação do `sw/README`	Com os valores de `sw/README` para T_i e T_j, os canais 0 e 1 geram exatamente a sequência de painéis que o coprocessador espera.	M	A
Barramentos e memórias				
BUS-01	Decodificação	Cada região alcançada pela porta certa; endereço fora do mapa gera access fault (`mcause` 1, 5 ou 7); a porta de dados não lê Boot ROM nem I-TCM.	D, M	A
BUS-02	Ponte OBI→AXI4	Acessos de 8, 16 e 32 bits no barramento de 128 b com lane e `WSTRB` corretos; non-posted (`rvalid` só depois de B ou R).	D, R, A	A
BUS-03	Ponte OBI→AXI4-Lite	Idem para registradores; SLVERR e DECERR viram `data_err_i`.	D, R, A	A
BUS-04	Interconnect AXI4	Arbitragem entre core e DMA na L2 sem inanição; acessos simultâneos aos bancos 0 e 1.	R, A, C	M
BUS-05	Memória de instrução	Boot ROM e o seletor entre Boot ROM e I-TCM mantêm a saída com `req=0`, como a I-TCM.	D, A	A
BUS-06	Memórias	D-TCM responde em 1 ciclo com byte enables; L2 de 64 KiB em dois bancos; Boot ROM inicializada.	D, M	M
Timer				
TMR-01	Contagem	`mtime` conta ciclos de `clk_i`; leitura de 64 bits coerente com o procedimento de software na virada dos 32 bits baixos.	D	M
TMR-02	Comparação	`irq_timer_i` em 1 enquanto `mtime` $\geq$ `mtimecmp`; cai ao escrever `mtimecmp` maior; escrita em duas partes sem disparo espúrio.	D, A	A
Sistema (`soc_top`)				
SYS-01	Boot	A Boot ROM ajusta a pilha para o topo da D-TCM e salta para `0x1000_0000`; `.rodata` na D-TCM.	D	A
SYS-02	Multiplicação de ponta a ponta	`matmul.c` com polling: C na L2 igual ao modelo, para vários M, N, K até o limite dos buffers (88×88).	D, M	A

ID	Feature	Comportamento esperado / critério	Método	Pri.
SYS-03	Fim por interrupção	O tratador vê `mcause = 0x8000_000B`, limpa `DONE/ERR` e retorna; o programa termina.	D, M	A
SYS-04	Vigia do timer	Um job mal configurado (`DIM_*` diferente da programação da DMA) é detectado pelo timer, e o software se recupera.	D	M
SYS-05	Tráfego concorrente	O core lê e escreve a L2 durante um job; o resultado não muda e nada trava.	R, M	M
SYS-06	Co-simulação no SoC	O Spike acompanha o core no `soc_top`; leituras de MMIO são impostas ao Spike.	M	M
SYS-07	Netlist	Conformal LEC equivalente para cada bloco sintetizado; gate-level com SDF de um subconjunto, quando houver biblioteca com modelos.	G	A

2.1.1 Pontos de atenção levantados na revisão do RTL

A leitura do RTL e das sínteses mais recentes apontou comportamentos que o plano deve confirmar ou que a equipe precisa decidir. Cada um vira teste dirigido, asserção ou item de cobertura.

P1. Dimensões fora do padrão no coprocessador. O número de tiles é `dim » 3`, com 0 virando 1. Com $M = 12$ e $N = 8$ o coprocessador envia 16 beats de C, mas o canal 2 da DMA, programado com $4 \cdot M \cdot N$ bytes, espera 24 e o job trava. Decidir entre exigir múltiplos de 8 (com um bit de erro em `STATUS`) ou completar com zeros (COP-10).

P2. Decisões do `coproc_regs` a confirmar. `START` aceito limpa `DONE`; offset fora do mapa responde SLVERR; `DIM_*` obedecem a `WSTRB` (COP-01, COP-02).

P3. Array parado durante a saída de C. Cada tile custa pelo menos $K + 30$ ciclos, e o pipeline do clr, que permitiria encadear tiles, não é usado pelo sequenciador. Não é erro; o teste COP-11 registra o número para o estudo de desempenho.

P4. `coproc_demux` sem uso. O módulo não é instanciado. Confirmar e remover da lista de arquivos, ou documentar a ligação que falta.

P5. Pedido de dados sem `instr_misaligned`. Para tirar a comparação de desvio do caminho `data_req_o` → `freeze` → `instr_req_o`, o `data_req_o` passou a usar só as exceções de load e store. A troca vale enquanto o decoder nunca marcar desvio ou JALR como acesso à memória: a asserção `mem_op_not_control` do `rv32_ex_stage` fica ligada em toda a regressão (CORE-13, CORE-19).

P6. Imediato do alvo em ID. O `rv32_branch_pred` monta os imediatos B, J e U direto dos bits da instrução, sem o decoder. Uma asserção em bind compara `id_target` com `ifid_pc` + `imm` do `rv32_imm_gen` para desvios, JAL e AUIPC, e a cobertura cruza os bits de imediato com os três formatos (CORE-11).

P7. Freeze inclui a espera de grant. O `freeze` agora é `mem_wait | req_wait`; os comentários de `rv32_hazard`, `rv32_id_stage` e a documentação ainda dizem só "espera no MEM". Cruzar latências de `gnt` e `rvalid` com redirects e traps no mesmo ciclo (CORE-12, CORE-13).

P8. Contrato da memória de instrução. O core depende de a memória manter a saída com `req=0`. O requisito está no README do core, mas não no de `rtl/top`; a Boot ROM e o seletor precisam cumprir o mesmo. O modelo de memória do testbench do core cumpre o contrato e uma asserção no `soc_top` o verifica (CORE-02, BUS-05).

P9. `DONE` do coprocessador não é o fim do job. O fim verdadeiro é o `DONE` do canal 2 da DMA. Os testes de sistema esperam por ele, e um teste dirigido mostra que ler C logo após o `DONE` do coprocessador pode pegar dados antigos (SYS-02).

P10. Job mal configurado trava sem erro. Se `DIM_*` e a programação da DMA discordarem, nenhum bloco acusa erro. O vigia do timer cobre o software (SYS-04); um contador de beats ou um bit de erro no coprocessador ajudaria a depurar.

P11. Registradores escritos à mão em três lugares. README, RTL e header C. Gerar os três, e também o modelo RAL do UVM, de uma fonte única (SystemRDL com PeakRDL, ou o reggen do OpenTitan) antes de escrever a DMA e o timer evita divergências que o RAL não pegaria.

P12. Caminhos combinacionais entre blocos. O `instr_req_o` depende de `data_gnt_i` e `data_rvalid_i` no mesmo ciclo. No `soc_top`, se o grant depender combinacionalmente do pedido, surge um caminho longo entre blocos; o lint e o `check_design` do Genus no `soc_top` devem mostrar que não há laço combinacional.

2.2 Estratégias de inserção de estímulos

O plano combina seis estratégias, da mais controlada à mais abrangente.

E1. Testes dirigidos. Programas curtos em assembly para o core (cada exceção, cada cenário de hazard, interrupção em posição conhecida) e sequências UVM dirigidas para os demais blocos (acesso a cada registrador, job 8×8 com $K = 1$, reset em cada estado).

E2. Conformidade. riscv-tests (`rv32ui`, `rv32mi`) e riscv-arch-test com RISCOF, comparando assinaturas com o Spike. No ambiente do core, um modelo de memória único atende às duas portas, para rodar os testes sem adaptar os linkers; o mapa Harvard real é verificado no `soc_top`.

E3. Aleatório de instruções. riscv-dv com perfis: geral, rico em dependências (hazards), rico em desvios, CSR e traps, loads e stores com desalinhamento. Cada programa roda em co-simulação com o Spike; várias sementes por perfil até fechar a cobertura.

E4. Aleatório restrito de transações. Sequências UVM com `rand` para dimensões e valores das matrizes (peso nos extremos), descritores da DMA, latências de `gnt` e `rvalid`, lacunas de `tvalid`, períodos de `tready` baixo, razão e fase dos clocks, e momento das interrupções.

E5. Programas de sistema. Os programas de `sw/tests` (boot, matmul com polling e com interrupção, timer) compilados e carregados no `soc_top`, com A e B geradas pelo modelo numpy na L2.

E6. Injeção de erros e casos negativos. Respostas de erro do barramento, endereços fora do mapa, instruções ilegais e desalinhadas, reset a qualquer momento, dimensões fora do padrão, `DIM_*` em desacordo com a DMA. O objetivo é mostrar recuperação determinística e documentar os limites de operação.

Verificação da resposta (checking).

- **Co-simulação com o Spike:** a cada instrução aposentada, o monitor RVFI entrega `pc`, instrução, `rd` e valor escrito, acesso à memória e trap. O Spike executa a mesma instrução e o scoreboard compara. Interrupções são informadas ao Spike pelo `mip` no ponto de commit; leituras de `mcycle` e de MMIO têm o valor imposto ao Spike.

- **Scoreboard do coprocessador:** compara cada beat do stream C com o modelo numpy e reporta a primeira divergência com tile, linha e coluna.
- **Preditor da DMA:** calcula a sequência de endereços e bursts de cada canal a partir dos registradores e compara com o monitor AXI4.
- **RAL:** `mirror` e `check` dos registradores do coprocessador, da DMA e do timer.
- **Asserções SVA (via `bind`):** protocolo OBI, AXI4, AXI4-Lite e AXI4-Stream; `instr_req_o == ~stall`; redirect implica IF/ID inválido no ciclo seguinte; trap limpa ID/EX e EX/MEM; nenhum `data_req_o` com `trap_i`; `mem_op_not_control`; alvo de ID igual ao do `rv32_imm_gen` (P6); um início e um fim de job por travessia no coprocessador; sem X nas saídas depois do reset.
- **Checagens de fim de teste:** `tohost` ou assinatura dos testes de conformidade; conteúdo de C na L2; nenhuma transação pendente nos agentes; FIFOs do coprocessador vazias.

2.3 Modelo de cobertura

Tabela 6: Grupos de cobertura funcional.

Covergroup	Coverpoints / bins principais
`cg_isa`	cada instrução; `rd = rs1`, `rd = x0`; operandos em 0, 1, −1, mínimo e máximo com sinal; deslocamento em 0 e 31; offset de load/store em cada byte.
`cg_hazard`	fonte do operando (banco, EX/MEM, MEM/WB, write-through) × operando (`rs1`, `rs2`, store, CSR); load-use; CSR-use; stall evitado por `rs*_ren`.
`cg_branch`	previsto × resolvido (tomado, não tomado); JAL; JALR; redirects simultâneos (trap+EX, EX+ID, trap+ID); redirect de ID durante stall; imediato com bit 1 em 1.
`cg_trap`	cada `mcause`; interrupção chegando com cada estágio ocupado; interrupção durante freeze; com acesso pendente; MRET com interrupção pendente.
`cg_obi`	latência de `gnt` (0, 1, 2–4, >4); latência de `rvalid`; erro em load e em store; resposta durante outro freeze.
`cg_csr`	cada CSR × operação (W, S, C) × forma (registrador, imediato) × fonte zero.
`cg_cop_cfg`	$M/8$ e $N/8$ (1, 2–4, >4); K (1, 2–7, 8–64, >64, máximo); um tile, uma linha de tiles, muitos tiles.
`cg_cop_flow`	lacuna em A, em B, nas duas; `c_tready` baixo em cada beat do tile; razão de clock (mais rápido, igual, mais lento); acesso a registrador com `BUSY`; jobs consecutivos.
`cg_dma`	`CNT1`, `CNT2` (1, pequeno, grande); stride 0; burst que tocaria 4 KiB; `LEN` (bins); canais simultâneos; erro em cada canal; `IRQ_EN` × `DONE/ERR`.
`cg_bus`	mestre × região; tamanho do acesso × lane; resposta (OKAY, SLVERR, DECERR); conflito core × DMA na L2.
`cg_sys`	cada programa de `sw/tests`; polling e interrupção; vigia disparado; core acessando a L2 durante o job.

A cobertura de código (linha, desvio, expressão, FSM e toggle) é medida por bloco e consolidada no IMC. Exclusões, como a IP de `vendor/` e o código só de simulação, ficam registradas com justificativa no repositório.

2.4 Resumo dos testes planejados

Tabela 7: Lista de testes e rastreabilidade com as features.

Teste	Nome	Estímulo	Features cobertas
Unidade			
T01	`core_units`	E1	testbenches `tb_rv32_*` e UVM `core_pc` existentes
T02	`coproc_units`	E1, E4	`coproc_mac`, `coproc_pe`, `coproc_skew`, `coproc_pe_array`; base de COP-03, COP-04
Core			
T03	`core_smoke`	E1	CORE-01, CORE-04
T04	`core_riscv_tests`	E2	CORE-04 a CORE-08, CORE-18
T05	`core_arch_test`	E2	CORE-04 a CORE-07, CORE-18, CORE-22
T06	`core_rand_isa`	E3	CORE-04, CORE-05, CORE-08, CORE-09, CORE-11
T07	`core_rand_pipeline`	E3, E4	CORE-09 a CORE-13, CORE-15
T08	`core_traps`	E1, E6	CORE-03, CORE-07, CORE-16 a CORE-19
T09	`core_irq`	E1, E4	CORE-20 a CORE-22
T10	`core_csr_counters`	E1	CORE-06, CORE-23
T11	`core_obi_stress`	E4, E6	CORE-02, CORE-13, CORE-15, CORE-17
T12	`core_perf`	E1	CORE-14
Coprocessador			
T13	`cop_regs`	E1, E4	COP-01, COP-02
T14	`cop_matmul_directed`	E1	COP-03 a COP-05
T15	`cop_matmul_rand`	E4	COP-03, COP-06, COP-07
T16	`cop_cdc`	E4	COP-08
T17	`cop_reset_negative`	E6	COP-09, COP-10
T18	`cop_perf`	E1	COP-11
DMA			
T19	`dma_regs`	E1, E4	DMA-01, DMA-06
T20	`dma_addr_bursts`	E4	DMA-02, DMA-03
T21	`dma_streams`	E1, E4	DMA-04
T22	`dma_errors`	E6	DMA-05, DMA-06
T23	`dma_coproc_pair`	E4	DMA-07, COP-03 (DMA e coprocessador juntos)
Barramentos e periféricos			
T24	`bus_decode_bridges`	E1, E6	BUS-01 a BUS-03, BUS-06
T25	`bus_xbar_rand`	E4	BUS-04
T26	`timer`	E1	TMR-01, TMR-02
Sistema			
T27	`soc_boot`	E5	SYS-01, BUS-05
T28	`soc_matmul_poll`	E5	SYS-02
T29	`soc_matmul_irq`	E5	SYS-03, DMA-06, CORE-20
T30	`soc_watchdog`	E5, E6	SYS-04, TMR-02
T31	`soc_concurrent`	E5, E4	SYS-05, BUS-04
T32	`soc_cosim`	E5	SYS-06
T33	`netlist_lec_gls`	—	SYS-07

3 Cronograma preliminar

Cronograma de 10 semanas a partir de 16/11/2026, sem contar o recesso de 21/12/2026 a 01/01/2027. A ordem acompanha o estado do RTL (Tabela 3): core e coprocessador primeiro, porque já existem; DMA, barramentos, memórias e timer conforme o RTL for entregue. Os marcos M1 a M4 coincidem com as versões v0.2 a v0.4 do projeto. Datas a ajustar com o orientador.

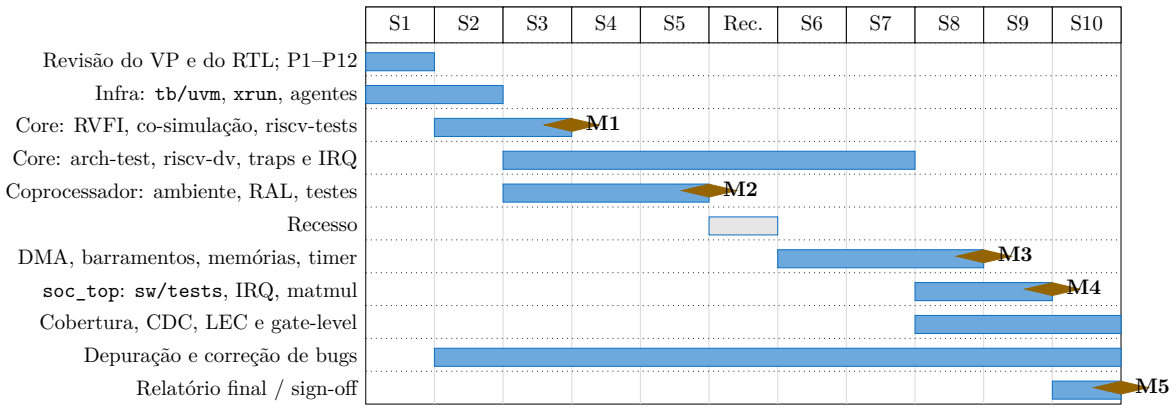


Figura 3: Cronograma preliminar (S1 = semana de 16/11/2026; "Rec." = recesso de fim de ano).

Tabela 8: Etapas, datas e entregáveis.

Semana	Período	Atividades e entregáveis
S1	16/11 – 20/11	Revisão do VP com o orientador e a equipe; decisões sobre P1–P12; lint de todo o RTL.
S1–S2	16/11 – 27/11	Estrutura de `tb/uvm`, Makefile e scripts `xrun`, agentes OBI, AXI4-Lite e AXI4-Stream, tracer RVFI do core.
S2–S3	23/11 – 04/12	Co-simulação Spike e riscv-tests no core. **M1 (v0.2):** riscv-tests aprovados em co-simulação.
S3–S5	30/11 – 18/12	Ambiente do coprocessador, RAL, modelo numpy, testes T13–T18. **M2:** coprocessador sem divergências na regressão aleatória.
S3–S7	30/11 – 15/01	Core: riscv-arch-test, riscv-dv, traps, interrupções, OBI com latências; cobertura do core.
S6–S8	04/01 – 22/01	DMA, barramentos, memórias e timer, conforme o RTL. **M3 (v0.3):** DMA e coprocessador aprovados juntos (T23).
S8–S9	18/01 – 29/01	`soc_top` com `sw/tests`. **M4 (v0.4):** matmul de ponta a ponta por polling e por interrupção.
S8–S10	18/01 – 05/02	Merge de cobertura no IMC, testes para os buracos, exclusões justificadas; CDC; LEC e gate-level.
S10	01/02 – 05/02	Relatório de verificação (resultados, cobertura, bugs). **M5:** sign-off segundo a Tabela 4.

Riscos ao cronograma.

(i) **RTL ainda não escrito** (DMA, timer, memórias, `soc_top`). Mitigação: agentes, modelos e preditores não dependem do RTL e são feitos antes; os testes desses blocos andam conforme as entregas.

(ii) **Infraestrutura nova:** tracer RVFI, co-simulação Spike, riscv-dv e RISCOF precisam

ser compilados e integrados no servidor do CEPEDI. Mitigação: começar na S1 e manter o caminho mínimo (riscv-tests com `tohost`) funcionando sem a co-simulação.

(iii) **Ambiguidades de especificação** (P1–P12) podem gerar retrabalho. Mitigação: resolver na S1.

(iv) **Gate-level:** a biblioteca tutorial do Genus não tem modelos de simulação; o gate-level depende da biblioteca do PDK. Mitigação: LEC a cada síntese até lá.

(v) **Tempo de simulação** dos jobs grandes e da regressão do core. Mitigação: regressão noturna; jobs grandes só no nível de bloco, onde não passam pelo core.

(vi) **Licenças** de Xcelium, IMC e Jasper. Mitigação: Verilator para lint e testes rápidos de unidade; CDC por revisão e simulação se não houver Jasper.

Referências

[1] A. Piziali. *Functional Verification Coverage Measurement and Analysis*. Springer, 2004.

[2] J. Bergeron, E. Cerny, A. Hunter, A. Nightingale. *Verification Methodology Manual for SystemVerilog*. Springer, 2005.

[3] Accellera. *Universal Verification Methodology (UVM) 1.2 User's Guide*, 2015; IEEE Std 1800.2-2020.

[4] RISC-V International. *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA* e *Volume II: Privileged Architecture*.

[5] H. T. Kung. Why systolic architectures? *IEEE Computer*, 15(1), 1982.

[6] Arm. *AMBA AXI and ACE Protocol Specification* e *AMBA AXI4-Stream Protocol Specification*.

[7] OpenHW Group. *OBI (Open Bus Interface) Specification*.